

Vesta – Functionality Test Scope & Feature Overview (UPDATED)

Version: 2.0

Date: January 2026

Platform: Android (Flutter/Dart client, Python backend)

Backend API: <https://backend-vesta.onrender.com>

Deep Link Domain: <https://vestaapp.co>

1. App Summary

Name: Vesta

Purpose: Personal Financial Management (PFM) app enabled by open banking.

Data Source: JOPACC Open Banking sandbox (test banks & test customers) + Firebase (auth & app data).

Core Capabilities

Vesta lets a user:

1. Onboard and create an account with phone verification
2. Link their bank accounts (via the JOPACC sandbox)
3. See consolidated balances & transactions
4. Categorize spending and assign it to "buckets"
5. Create a personal budget plan with customizable cycles
6. Track spending vs budget with real-time analytics
7. Create a shared "Household" and track a joint budget
8. Set and track savings goals

This document describes the functional scope for QA / functionality testing. A separate section at the end describes the test customers and accounts available in the JOPACC sandbox.

2. Onboarding & Authentication Flow

2.1 First-Time User Journey

Step 1: Launch & Sign Up

- Launch app → Welcome / Sign Up screen
- User chooses **Sign Up**
- Required fields:
 - **Username** (alphanumeric and underscores only, regex: `^[a-zA-Z0-9_]+$`)
 - **Email** (standard email format, regex: `^[\\w-\\.]+@([\\w-]+\\.)+[\\w-]{2,4}$`)
 - **Password** + repeat password
- **Validation:**
 - All fields required
 - Passwords must match
 - Username format validation (alphanumeric + underscore only)
 - Email format validation
 - Duplicate username/email check against Firestore `usernames` and `emails` collections
 - Fallback check in `users` collection if new collections don't exist

Step 2: Phone Number Verification (OTP)

- Screen: "Enter your phone number"
- User enters Jordanian mobile number in format: `7XXXXXXXX`
- App prepends `+962` automatically
- Taps **Send** button
- **Firestore Phone Auth:**
 - Sends OTP code via SMS
 - **Test Environment:** You can use a real phone number or this test phone number `0799999999` with OTP `291204` (Firestore test number)
 - **Production:** Real SMS sent to actual phone numbers
- User enters the 6-digit OTP on verification screen
- **Resend OTP:** User can tap "Resend OTP" if code not received within 60 seconds
- On OTP verification:
 - Phone credential is verified with Firebase
 - Email credential is linked to phone credential
 - **Critical Race Condition Prevention:**
 - Final username/email uniqueness check performed AFTER phone auth

- Batched writes atomically create:
 - `usernames/{username.toLowerCase()} document`
 - `emails/{email.toLowerCase()} document`
 - `users/{uid} document`
- If batch fails (duplicate detected), Firebase Auth user is deleted
- Prevents race conditions during signup
- Display name is set to username
- User document created with initial fields: {


```
username, email, phoneNumber,
currency: "JOD",
totalBalance: 0.0,
totalIncome: 0.0,
totalExpense: 0.0,
dayOfMonth: 28, // Budget reset day
householdIds: [],
createdAt: serverTimestamp
}
```
- **Default Categories Initialization:**
 - System creates 4 default spending categories:
 - Food And Drinks (expense)
 - Groceries (expense)
 - Entertainment (expense)
 - Others (expense)

Step 3: Profile Setup

- User can set:
 - **Monthly total income** (can be changed later in Personal Budget section or main screen)
 - **Default budget reset day:** 28th of the month (used for spending cycles)
 - This can be customized later from settings
 - Stored in `dayOfMonth` field

Step 4: Login

- After signup, user is logged in automatically
- Subsequent sessions use **email + password** login (Firebase Auth)
- **Note:** Login also accepts username instead of email
 - App looks up username in `usernames` collection to get email
 - Then authenticates with email + password
- Standard error messages for invalid credentials, network errors, etc.

- **Budget Cycle Finalization on Login:**

- On each login, `handleBudgetCycleOnLogin()` is called
- System checks if a budget cycle has ended
- Finalizes previous cycle with actual spending/savings from transactions
- Creates new cycle document if needed with expected values
- This is an automated background process

Step 5: Deep Link Handling for Household Invites

- If the app was opened via a deep link (e.g., `https://vestaapp.co/join?inviteId=...`) while the user was logged out:
 - The link is stored internally in `_pendingInviteLink`
 - After login, `checkPendingLink()` is called
 - Vesta shows a dialog asking if the user wants to accept the household invitation
 - **Edge Cases:**
 - Users cannot accept their own invitations (self-invite prevention)
 - Invalid or expired invite IDs show error message
 - Deep link service prevents duplicate initialization

2.2 Validation Rules Summary

Field	Rule	Error Message
Username	Alphanumeric + underscore only	"Username can only contain letters, numbers, and underscores"
Username	Must be unique	"Username already taken. Please choose another one."
Email	Valid email format	"Please enter a valid email address"
Email	Must be unique	"Email already registered. Please use another email or login."
Password	Must match repeat password	"Passwords do not match"

Field	Rule	Error Message
Phone	Jordanian format (7XXXXXXX)	Validation in phone number page
OTP	6 digits	"Please enter a 6-digit OTP"

3. Accounts & Transactions

3.1 Bank Account Linking (JOPACC Sandbox)

Linking Flow

1. From the Accounts or Home screen, user taps "Link Bank Account"
2. User sees **ChooseBankSplash** screen
3. User selects bank: **Bank of JoPACC** (currently only supported bank)
4. **JopaccLinkScreen** - 3-step process:
 - **Step 1: Login**
 - User enters JoPACC sandbox username and password
 - Credentials stored in `providers.jopacc.username` field
 - **Step 2: Consent**
 - User sees consent screen for data access
 - Permissions requested:
 - Read account details and balances
 - Read transactions
 - Read standing orders
 - **Step 3: Account Selection**
 - Backend calls JOPACC API to fetch accounts
 - User selects which accounts to link
 - Accounts stored in `users/{uid}/accounts` collection with `linked: true` status

Backend Integration

- **Backend Endpoint:** `https://backend-vesta.onrender.com/sync_accounts/{uid}/{username}`
- App calls backend which interfaces with JOPACC Open Banking API
- Accounts are stored with JoPACC account IDs as document IDs

3.2 Accounts Screen

Display

- Lists all linked accounts for the logged-in user
- **Filter:** Only shows accounts where `linked: true`
- For each account:
 - Bank name (e.g., "Bank of JoPACC")
 - Account type (savings/current/salary/checking/payroll/merchant/operating)
 - Masked account identifier (e.g., •••• 3234 - last 4 digits of IBAN)
 - Balance (JOD)
 - Status flags where applicable:
 - Account status: active/suspended/closed
 - Lock status: credit locked, debit locked
- **Total Balance:**
 - Shown at top of Home screen
 - Sum of all linked active accounts
- **Add Account Button:**
 - FAB (Floating Action Button) to link new accounts
 - Redirects to bank linking flow

3.3 Account Sync

Sync Process

- User taps "Link Bank Account" or "Sync Accounts"
- Mobile app calls: `GET /sync_accounts/{uid}/{username}`
- **Backend behavior:**
 - Calls JOPACC sandbox API to fetch account list
 - Stores/updates account documents in Firestore
 - Account data includes:
 - `accountId` (unique identifier)
 - `accountType`
 - `balance` (object with amount and currency)
 - `status` (active/suspended/closed)
 - `creditLocked` (boolean)
 - `debitLocked` (boolean)
 - `linked` (boolean - set to true for linked accounts)

- Bank details from API response

3.4 Transaction Sync

Sync Trigger

- User taps "Sync transactions" from Accounts or Transactions screen
- Alternatively, automatically triggered after account linking
- Mobile app calls: `GET /get_transactions/{uid}/{accountId}` for each linked account

Backend Transaction Processing

- **Backend endpoint:** `/get_transactions/{uid}/{accountId}`
- Backend calls JOPACC sandbox API: `/account_id/transactions`
 - Note: JOPACC API returns all transactions (no pagination parameters in MVP)
- Backend flattens each transaction into standard structure:

```
{
  accountId: string,
  amount: number,
  currency: string (e.g., "JOD"),
  type: "debit" | "credit",
  date: ISO string (settlementDateTime),
  merchantName: string (from creditor/debtor data),
  description: string (from remittance info),
  accountLabel: string (last 4 digits of IBAN),
  category: null, // Initially null - user must assign
  source: "openBanking"
}
```
- **Idempotency (Backend behavior):**
 - For each transaction ID:
 - If already exists: only updates fields, **preserves existing user-assigned category**
 - If new: creates `transactions/{transactionId}` document
 - This ensures re-running sync doesn't duplicate transactions or erase categories

Transaction Sources

Transactions can originate from three sources:

1. **Open Banking** (`source: "openBanking"`) - Synced from JoPACC API
2. **SMS Paste** (`source: "sms"`) - User pastes transaction details from bank SMS
3. **Manual Entry** (`source: "manual"`) - User manually creates transaction

3.5 Standing Orders / Scheduled Payments Sync

Overview

- **Standing Orders (SOSPs)** are recurring scheduled payments
- Fetched from JOPACC Open Banking API
- Displayed in Personal Budget screen as "Upcoming Payments"

Sync Process

- Mobile app calls: `GET /get_sosps/{uid}/{accountId}` for each linked account
- Backend calls JOPACC API to fetch standing orders
- Stored in Firestore under account sub-collection
- Data includes:
 - Beneficiary information
 - Payment amount and currency
 - Next payment date
 - Status (active/inactive)
 - Frequency (monthly, weekly, etc.)

3.6 Transactions Screen

Main View

- Main "Transactions" tab shows list of all transactions from all linked accounts
- Sorted by date (newest first)

Filters

- **Account Filter:** "All Accounts" or specific account
- **Category Filter:** "All Categories" or specific category

Transaction Card Display

Each transaction card shows:

- **Amount:**
 - Green for credit (incoming)

- Red for debit (outgoing)
- Currency symbol (JOD)
- **Masked account label:** e.g., **** 3234
- **Date:** Settlement date
- **Merchant name:** Extracted from transaction data
- **Category dropdown:**
 - User can select category (Food and Drinks, Groceries, Entertainment, Others, Clothing, etc.)
 - Initially empty for Open Banking transactions
- **"Assign" button:**
 - Visible if user is member of at least one household
 - Assigns transaction to a household for shared budget tracking

Transaction Details

- Tap on transaction to see full details:
 - Full description
 - Merchant information
 - Transaction ID
 - Account details
 - Source (Open Banking, SMS, Manual)

3.7 Add Transaction (Manual Entry)

Access

- "+" button on Transactions screen

Fields

- **Amount:** Numeric input
- **Type:** Debit or Credit toggle
- **Merchant:** Text input
- **Notes:** Text area
- **Date:** Date picker
- **Account:** Dropdown of linked accounts
- **Category:** Category dropdown

Validation

- Amount must be greater than 0
 - Account must be selected
 - Date cannot be in future
-

4. Budgeting & Spending Analysis

4.1 Plan Budget (Setup Screen)

Purpose

- Define monthly budget plan using percentage allocations
- Set expected income for the budget cycle

User Inputs

- **Total monthly income (JOD)**
 - Note: This is **planned/expected income**, not tracked from actual deposits
- **Three percentage allocations:**
 - **Savings %** - Portion allocated to savings goals
 - **Essential Spending %** - Necessities (rent, groceries, bills, food)
 - **Luxuries & Entertainment %** - Non-essential spending

Validation

- Each field must be between 0–100%
- Sum of the three percentages cannot exceed 100%
- System allows sum < 100% (unallocated buffer)

Data Storage

- Stored per budget cycle as: users/{uid}/budget/{YYYY-MM}:
 - income: number,
 - saving: number (percentage),
 - spending: number (percentage, essentials),
 - luxuries: number (percentage),
 - currency: "JOD",
 - startDate: Timestamp,

endDate: Timestamp,
createdAt: serverTimestamp,
updatedAt: serverTimestamp

- Also updates `users/{uid}.totalIncome` for reuse elsewhere

4.2 Personal Budget Screen

Overview Section

For the selected budget cycle, shows:

- **Line chart** displaying 6-month trend:
 - **Budget Plan line** (dashed grey) - Total budget per cycle
 - **Total Spending line** (solid blue) - Essential + Luxury spending
 - **Savings line** (solid green) - Actual savings transfers
 - X-axis: Last 6 cycles
 - Y-axis: JOD amounts

Budget Plan Card

- Displays for current cycle:
 - **Savings allocation:** JOD amount ($\text{income} \times \text{saving\%}$)
 - **Essential Spending allocation:** JOD amount ($\text{income} \times \text{spending\%}$)
 - **Luxuries allocation:** JOD amount ($\text{income} \times \text{luxuries\%}$)
- Based on user's income and percentages set in Plan Budget

Upcoming Payments

- Lists active Standing Orders (SOSPs) with:
 - Beneficiary name
 - Next payment date
 - Amount (JOD)
 - Status indicator (active/inactive)
- Shows next 3-5 upcoming scheduled payments

Summary Components

- **Income Received Tracker:**
 - Shows expected income vs actual income received (if tracked)

- Progress bar visualization
- **Budget vs Actual:**
 - Current cycle spending compared to total budget
 - Over/under budget indicator

4.3 Budget Cycles & Time Windows

Budget Cycle Logic

The app uses a **customizable budget cycle** defined by a reset day (default: 28th).

Cycle Calculation:

- User's reset day is stored in `users/{uid}.dayOfMonth` (default: 28)
- **If today's day $\geq$ reset day:**
 - Cycle runs from: `{current month, reset day} → {next month, reset day - 1}`
 - Example (reset day = 28): Jan 28 → Feb 27
- **If today's day $<$ reset day:**
 - Cycle runs from: `{previous month, reset day} → {current month, reset day - 1}`
 - Example (reset day = 28, today = Jan 15): Dec 28 → Jan 27

Label Month:

- Budget documents are labeled by the month they END in
- Example: Cycle Dec 28 → Jan 27 is labeled as "2026-01"

Important Note:

- Budget cycles are NOT calendar months
- Always refer to them as "budget cycles" not "monthly budgets"
- A cycle can span across two calendar months

Cycle Consistency

This logic is used consistently across:

- Personal Budget spending tracker
- Spending Analysis screen
- Household spending tracking
- Budget finalization on login

Budget Cycle Finalization (Automated)

Trigger: Every login via `handleBudgetCycleOnLogin()`

Process:

1. **Determine current and previous cycle dates** based on `dayOfMonth`
2. **Finalize previous cycle** (if budget document exists):
 - Query all transactions within previous cycle date range
 - Calculate actual spending by category-bucket mapping
 - Calculate actual savings transfers
 - Update previous cycle document: {
 - actualSpending: number,
 - actualSavings: number,
 - startDate: Timestamp,
 - endDate: Timestamp,
 - updatedAt: serverTimestamp
3. **Initialize current cycle** (if doesn't exist):
 - Copy forward percentages from last known plan
 - Use current `totalIncome` value
 - Create new cycle document with expected values
 - Set startDate and endDate timestamps

Category-to-Bucket Mapping (for finalization):

Category	Bucket	Counted in Spending
Groceries	Essential	Yes
Food And Drinks	Essential	Yes
Entertainment	Luxury	Yes

Category	Bucket	Counted in Spending
Others	Luxury	Yes
Savings	Savings	No (separate tracker)
(uncategorized)	-	No

Data Sources:

- Reads from: `users/{uid}/accounts/{accountId}/transactions`
- Filters by: date range (cycle start to end exclusive)
- Only counts transactions with assigned categories

4.4 Spending Analysis Screen

Spending Budget Bar

- Shows for **current cycle**:
 - **Top value:** Current cycle spending (JOD)
 - Sum of essential + luxury spending
 - **Bottom value:** Budget amount (JOD)
 - Sum of $(\text{income} \times \text{spending}\%) + (\text{income} \times \text{luxuries}\%)$
 - **Progress bar:** Uses `currentSpending / totalBudget` ratio
 - Color indicators:
 - Green: Under budget
 - Yellow: Approaching budget (>80%)
 - Red: Over budget (>100%)

Spending Calculation

Spending is calculated by:

1. Taking all **debit** transactions within current cycle dates
2. **Ignoring** transactions without a category
3. Mapping each category to a bucket:
 - **Essential bucket:** Groceries, Food And Drinks, Bills, Rent
 - **Luxury bucket:** Entertainment, Others, Dining Out, Subscriptions
 - Anything else is ignored for budget tracking
4. Summing amounts for essential + luxury categories

Spending Categories Section

- Shows **top 3-5 categories** by absolute amount
- Each category row displays:
 - **Category icon & name**
 - **Color-coded circle** (visual identifier)
 - **Number of transactions** in current cycle
 - **Net amount:**
 - Red with "-" prefix for debits
 - Green with "+" prefix for credits
 - JOD currency
- Tap "See All" to view full category breakdown

Latest Transactions Section

- Lists **5 most recent transactions** across all accounts
- Same card format as main Transactions screen
- When user changes category on a transaction:
 - Summary and budget bar recalculate immediately
 - Real-time reactivity using Firestore streams

Time Period Filters

- **Day:** Today only
 - **Week:** Last 7 days
 - **Month:** Current calendar month
 - **Year:** Current calendar year
 - **Cycle:** Current budget cycle (most common)
-

5. Household Shared Budget

5.1 Household List & Creation

Households Screen

- Shows all households the user is a member of
- Reads from `users/{uid}.householdIds` array

- If empty: shows empty state with "Create Household" button
- If populated: lists household cards

Create Household Flow

1. User taps "Create Household" or FAB "+" button
2. **CreateHousehold screen:**
 - Enter **household name** (required)
3. On submit:
 - Creates `households/{householdId}` document:


```
{
  householdName: string,
  createdBy: uid,
  members: [uid], // Creator added automatically
  budget: 0, // Manual budget amount (flat value, not percentage)
  createdAt: serverTimestamp
}
```
 - **Batched write** atomically:
 - Creates household document
 - Adds household ID to `users/{uid}.householdIds` array
4. Redirects to Household Detail page

Household List Display

Each household card shows:

- Household name
- Member count
- Tap to open Household Detail page
- **Long press to delete:**
 - Shows confirmation dialog
 - On confirm:
 - Removes household ID from `users/{uid}.householdIds`
 - Deletes `households/{householdId}` document
 - **Warning:** Irreversible action

5.2 Inviting Members (Deep Links)

Invitation Flow

1. From Household Detail page, user taps **"Invite Member"** icon (person_add_alt_1)
2. App creates `invites/{inviteId}` document:

```
{
  householdId: string,
  inviterUid: string,
  inviterName: string (display name or email),
  householdName: string,
  status: "pending",
  createdAt: serverTimestamp
}
```
3. Deep link generated: `https://vestaapp.co/join?inviteId={inviteId}`
4. Share dialog opens (via `share_plus` package)
 - Message: "Join my household '{householdName}' on Vesta! Click here: {inviteLink}"
 - User can share via any installed app (WhatsApp, SMS, Email, etc.)

Receiver's Flow - Not Logged In

1. User taps deep link while logged out
2. **Deep Link Service** (`DeepLinkService`) detects no authenticated user
3. Link stored in `_pendingInviteLink`
4. User is sent to login/onboarding screens
5. **After successful login:**
 - `checkPendingLink()` is called
 - System retrieves stored invite link
 - Dialog shown: "You're Invited!"
 - Message: "{InviterName} has invited you to join {HouseholdName}"
 - Buttons: "Decline" | "Accept"

Receiver's Flow - Already Logged In

1. User taps deep link while logged in
2. **Deep Link Service** immediately shows "You're Invited!" dialog
3. Same dialog as above (inviter name, household name, Accept/Decline)

Accepting Invitation

1. User taps "Accept" button
2. App updates invite document:`invites/{inviteId}`:

```
status: "accepted",
acceptedByUid: {currentUserUid}
```
3. **Cloud Function (backend)** triggered by status change:

- Adds user UID to `households/{householdId}.members` array
- Adds household ID to `users/{uid}.householdIds` array
- Both updates ensure user becomes household member
- 4. Success message: "Invite accepted! Joining household..."
- 5. User can now view household in their Households list

Edge Cases & Security

- **Self-Invite Prevention:**
 - If `inviterUid === currentUserUid`, show error: "You can't accept your own invite."
- **Invalid Invite ID:**
 - If invite document doesn't exist: "Invite link is invalid or expired."
- **Expired Invites:**
 - Currently no expiration logic (future enhancement)
- **Duplicate Acceptance:**
 - Cloud Function should check if user already in members array

Deep Link Technical Details

- **Package:** `app_links` (Flutter plugin)
- **Domain:** `vestaapp.co`
- **Path:** `/join`
- **Query Parameter:** `inviteId`
- **State Management:**
 - Singleton service: `DeepLinkService`
 - Initialized once in main app widget
 - Handles both cold start (app terminated) and warm links (app running)
 - Prevents duplicate initialization with `_isInitialized` flag

5.3 Household Detail Screen

Header Section

- Household name (editable by members)
- **Member Avatars:**
 - Horizontal scrollable list
 - Shows first letter of username in colored circle

- Displays username below avatar
- Fetched via Firestore stream: `users` collection where `documentId` in `members` array

Spending Overview Chart

- **Line chart** showing last 6 cycles:
 - **Blue solid line:** Total household spending per cycle (essential + luxury)
 - **Grey dashed line:** Household budget per cycle
 - X-axis: Cycle months (last 6)
 - Y-axis: JOD amounts
- Data aggregated from `households/{householdId}/transactions`

Budget Tracker Bar

- Shows **current cycle** spending vs household monthly budget
- **Tap to update budget:**
 - Opens input dialog
 - User enters manual budget amount (JOD)
 - **Important:** Household budgets are **flat JOD amounts**, NOT percentages
 - Saves to `households/{householdId}.budget`
 - Triggers recalculation of chart and tracker bar
- Progress bar:
 - Top: Current spending (JOD)
 - Bottom: Budget amount (JOD)
 - Color coding same as personal budget

Spending Categories

- Similar to personal spending categories
- Scoped to **household transactions only**
- Shows top 5 categories by net amount
- Each category shows:
 - Icon and name
 - Transaction count (within current cycle)
 - Net amount (debit/credit sum)
- Tap "See All" to view full breakdown

Latest Transactions

- Last 5 transactions assigned to this household
- Shows:
 - Merchant name
 - Amount (red for debit, green for credit)
 - Date
 - Category
 - **Assigned by:** Username of member who assigned it
- Tap "All Transactions" to view full list (filtered by household)

Actions

- **Invite Member:** Shown in app bar (person_add icon)
- **View All Transactions:** Link to transactions screen filtered by household

5.4 Assigning Transactions to a Household

Visibility

- On any Transaction card in global Transactions screen
- If user is member of **at least one household**:
 - "Assign" button is visible

Assignment Process

1. User taps "Assign" button on transaction card
2. Bottom sheet opens showing list of user's households
3. User selects which household to assign transaction to
4. Backend creates copy of transaction in household:households/{householdId}/transactions/{transactionId}:

```
{
  ...all transaction fields...,
  assignedBy: currentUserUid,
  assignedAt: serverTimestamp
}
```
5. Transaction now appears in household's:
 - Latest Transactions
 - Spending Categories
 - Budget calculations
 - Spending chart data

Data Structure

- **Original transaction:** Stays in `users/{uid}/accounts/{accountId}/transactions`
- **Household transaction:** Copy in `households/{householdId}/transactions`
- Both share same transaction details
- Household copy includes `assignedBy` field for auditing

Member Visibility

- All household members can see all assigned transactions
 - Household spending analysis aggregates transactions assigned by any member
 - Promotes transparency in shared budget management
-

6. Savings

6.1 Savings Screen

Overview

- Lists all user's savings goals
- Shows progress for each goal (current/target)
- **Empty state:** If no goals, shows "Create Savings Goal" prompt

Display

Each savings goal card shows:

- **Goal emoji** (visual identifier)
- **Goal title** (e.g., "New Car", "Vacation")
- **Current amount** (JOD)
- **Target amount** (JOD)
- **Progress bar:** current / target percentage
- **Remaining amount:** target - current
- **Actions:** Allocate/Deallocate buttons

6.2 Create Savings Goal

Pre-defined Templates

- New Car 🚗
- Vacation 🌴
- House 🏠
- Emergency Fund 🆘
- Custom Goal (user-defined emoji and title)

Fields

- **Goal Title:** Text input (or selected template)
- **Target Amount:** Numeric input (JOD)
- **Emoji:** Selected from template or custom picker

Data Storage

```
users/{uid}/savings/{goalId}:  
  goalTitle: string,  
  currentAmount: 0, // Initially zero  
  targetAmount: number,  
  emoji: string,  
  createdAt: serverTimestamp
```

6.3 Savings Allocation

Total Savings Tracking

- User's total savings stored in: `users/{uid}.totalSavings`
- **Important:** This is a **manual value**, NOT derived from account balances
- User must manually update total savings amount

Allocation Process

1. User taps "Allocate" on a savings goal
2. System calculates **available amount**:
 - `available = totalSavings - (sum of all currentAmount across all goals)`
3. User enters amount to allocate (cannot exceed available)

4. Updates goal's `currentAmount`
5. Allocated amount is now reserved for that specific goal

Deallocation

- User can tap "Deallocate" to remove funds from a goal
- Returns amount to available pool
- Does NOT change `totalSavings` (only reallocates within existing savings)

Manual Nature

- **No automatic integration with bank accounts** in MVP
 - User must manually:
 - Update total savings when they transfer money to savings account
 - Allocate those savings to specific goals
 - Future enhancement: Derive from designated "savings" account balances
-

7. Backend API Endpoints

7.1 Base URL

```
https://backend-vesta.onrender.com
```

7.2 Account Sync

Endpoint: `GET /sync_accounts/{uid}/{username}`

Purpose: Sync user's accounts from JOPACC Open Banking API

Parameters:

- `uid`: Firebase user ID
- `username`: JOPACC sandbox username

Process:

1. Backend authenticates with JOPACC API
2. Fetches account list for the username
3. Stores/updates accounts in Firestore: `users/{uid}/accounts`
4. Sets `linked: true` for all fetched accounts

Response:

```
{
  "status": "success",
  "accounts": [...],
  "count": number
}
```

7.3 Transaction Sync

Endpoint: `GET /get_transactions/{uid}/{accountId}`

Purpose: Fetch and sync transactions for a specific account

Parameters:

- `uid`: Firebase user ID
- `accountId`: JOPACC account identifier

Process:

1. Backend calls JOPACC API: `{accountId}/transactions`
2. Flattens transaction data to standard format
3. For each transaction:
 - If exists in Firestore: update fields (preserve category)
 - If new: create transaction document
4. Stores in: `users/{uid}/accounts/{accountId}/transactions`

Response:

```
{
  "status": "success",
  "transactions": [...],
}
```



```
"count": number,  
"new": number,  
"updated": number  
}
```

Idempotency:

- Safe to call multiple times
- Preserves user-assigned categories
- No duplicates created

7.4 Standing Orders Sync

Endpoint: GET /get_sosps/{uid}/{accountId}

Purpose: Fetch standing orders (scheduled payments) for an account

Parameters:

- uid: Firebase user ID
- accountId: JOPACC account identifier

Process:

1. Backend calls JOPACC API for standing orders
2. Stores in Firestore sub-collection
3. Used to display "Upcoming Payments" in budget screen

Response:

```
{  
  "status": "success",  
  "sosps": [...],  
  "count": number  
}
```

7.5 Error Handling

All endpoints return standard error format:

```
{
  "status": "error",
  "message": "Human-readable error message",
  "code": "ERROR_CODE"
}
```

Common error codes:

- `AUTH_FAILED`: JOPACC authentication failed
 - `ACCOUNT_NOT_FOUND`: Account ID not found in JOPACC
 - `INVALID_UID`: Firebase UID not found
 - `NETWORK_ERROR`: Connection to JOPACC API failed
-

8. Test Data – JOPACC Sandbox Customers & Accounts

8.1 Customer Distribution

Individual Customers (36 accounts total)

Single Account Customers (8 customers, 8 accounts)

- Customer IDs: `IND_CUST_001` to `IND_CUST_008`
- Account types distributed: savings, current, salary

Two Account Customers (6 customers, 12 accounts)

- Customer IDs: `IND_CUST_009` to `IND_CUST_014`
- Each has: 1 savings + 1 current account

Three Account Customers (4 customers, 12 accounts)

- Customer IDs: IND_CUST_015 to IND_CUST_018
- Each has: 1 savings + 1 current + 1 salary account

Custodian Accounts (4 customers, 4 accounts)

- Customer IDs: IND_CUST_019 to IND_CUST_022
- All are savings accounts with custodian information
- Relationship types: guardian / parent

Corporate Customers (16 accounts total)

Single Account Customers (4 customers, 4 accounts)

- Customer IDs: CORP_CUST_001 to CORP_CUST_004
- Account types: checking, savings, payroll

Two Account Customers (3 customers, 6 accounts)

- Customer IDs: CORP_CUST_005 to CORP_CUST_007
- Each has: 1 checking + 1 payroll account

Three Account Customers (2 customers, 6 accounts)

- Customer IDs: CORP_CUST_008 to CORP_CUST_009
- Each has: 1 checking + 1 savings + 1 payroll account

Business Customers (10 accounts total)

Single Account Customers (4 customers, 4 accounts)

- Customer IDs: BUS_CUST_001 to BUS_CUST_004
- Account types: merchant, operating

Two Account Customers (3 customers, 6 accounts)

- Customer IDs: BUS_CUST_005 to BUS_CUST_007
- Each has: 1 merchant + 1 operating account

8.2 Transaction Data

Each test account has synthetic transaction history including:

- **Transaction Types:**
 - Debits (outgoing payments)
 - Credits (incoming payments)
 - Internal transfers
 - Standing orders
- **Merchant Variety:**
 - Retail purchases
 - Utility payments
 - Salary deposits
 - ATM withdrawals
 - Online payments
- **Date Ranges:**
 - Historical data spanning multiple months
 - Enables testing of budget cycles across time periods

8.3 Test Credentials

Format: To be provided separately for security

Expected structure:

Customer ID	Username	Password	Account Types	Notes
IND_CUST_001	[TBD]	[TBD]	Savings	Active, normal balance
IND_CUST_002	[TBD]	[TBD]	Current	Suspended, credit locked
...	...	...	...	...

Note: Actual credentials should be provided in a separate secure document or test environment dashboard.

9. Out of Scope / Important Notes for Testers

9.1 Read-Only Financial Data

- **No real bank data** is used; only JOPACC sandbox and Firebase test environment
- App is **read-only** for financial data:
 - Cannot initiate new payments
 - Cannot perform money transfers between accounts
 - Cannot execute new card transactions
- Historical transaction data (including card payments) is displayed, but these are read-only records from the sandbox

9.2 Known Limitations

- **Single Bank Support:** Only Bank of JoPACC is supported in MVP
- **Manual Savings:** No automatic derivation from account balances
- **No Transaction Search:** Cannot search transactions by keyword (only filter by account/category)
- **Limited Budget History:** Charts show last 6 cycles maximum
- **No Biometric Auth:** Login requires email/password every time (no fingerprint/face ID)

9.3 Testing Environment Details

Firestore:

- Test phone number: 0799999999 with OTP 291204
- Real SMS sent in production environment for other numbers

Backend:

- Hosted on Render (may have cold start delays on first request after inactivity)
- Expect 5-10 second initial response time if backend is idle

Data Persistence:

- All data persists in Firestore
- Testers should use unique usernames/emails to avoid conflicts
- Test data can be manually cleared from Firebase console if needed

9.5 Test Account

Pre-configured test account for quick access:

- **Username:** Abdullah
- **Email:** abdullah.omarr04@gmail.com
- **Password:** 111111
- **Status:** Already linked to JOPACC accounts with transaction history
- **Use:** For quick exploratory testing without going through onboarding